

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL PERSONALIZADO
(NIVEL 4 (INTEGRADOR))

Track 01: Aplicaciones Web con Python (FastAPI & Streamlit)

«El Restaurante: La Carta (Frontend) y la Cocina de Alta Eficiencia (Backend)»

Arquitectura de aplicaciones web full-stack en Python: Backend asíncrono de alto rendimiento
con FastAPI, documentación Swagger automática y Frontend interactivo con Streamlit.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Integrador / Producción | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Arquitectura Cliente-Servidor y APIs Modernas	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Diagrama de Flujo Full-Stack: Streamlit <-> FastAPI <-> DB	Pág. 5
Capítulo 3	Implementación Práctica en Python: Backend FastAPI con Endpoint RESTful Tipado	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas y Seguridad en Aplicaciones Web	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la separación de responsabilidades Cliente-Servidor, APIs RESTful y el paradigma asíncrono async/await.



Competencia Práctica

Construir y desplegar una aplicación web completa con endpoints RESTful, validación con Pydantic y dashboard interactivo.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Arquitectura Cliente-Servidor y APIs Modernas

Una aplicación web desacoplada divide la presentación visual del procesamiento central de datos mediante contratos de comunicación HTTP (APIs REST).

☀ **Metáfora Central: El Restaurante: La Carta (Frontend) y la Cocina de Alta Eficiencia (Backend)**

El frontend (Streamlit) es la carta elegante y el mozo que atiende al comensal en la mesa. El backend (FastAPI) es la cocina profesional donde los chefs procesan las comandas con máxima higiene, rapidez y orden, entregando los platos listos en formato JSON.

Principios Teóricos y Modelo Mental

FastAPI: Framework moderno, basado en Starlette y Pydantic, con soporte nativo de asincronía (ASGI) y tipado estático.

Verbos HTTP Semánticos: GET (consultar datos), POST (crear nuevos registros), PUT (actualizar), DELETE (eliminar).

⚡ **Regla de Oro en Python**

Nunca mezcles lógica pesada de base de datos en el cliente visual; el cliente solo consume y renderiza.

2. Diagrama de Flujo Full-Stack: Streamlit <-> FastAPI <-> DB

Comunicación asíncrona mediante peticiones HTTP/JSON y validación cruzada.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	El usuario interactúa con widgets en Streamlit y presiona un botón.	Evento en UI
2. Evaluación	Streamlit envía una petición HTTP POST /api/v1/recurso con payload JSON.	Request sobre HTTP
3. Transformación	FastAPI valida los datos con Pydantic, ejecuta la lógica y persiste en DB.	Validación & Persistencia
4. Retorno / Salida	FastAPI responde HTTP 201 Created y Streamlit actualiza la vista reactivamente.	UI actualizada

Visualización Mental

FastAPI genera automáticamente documentación Swagger interactiva en la ruta /docs para probar todos tus endpoints.

3. Backend FastAPI con Endpoint RESTful Tipado

Servicio web profesional con validación de modelos y control de estado HTTP:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

app = FastAPI(title="API de Gestión de Productos", version="1.0.0")

class ProductoDTO(BaseModel):
    nombre: str
    precio: float
    categoria: str

db_productos: list[dict] = []

@app.post("/productos", status_code=status.HTTP_201_CREATED)
async def crear_producto(prod: ProductoDTO):
    nuevo = {"id": len(db_productos) + 1, **prod.model_dump()}
    db_productos.append(nuevo)
    return {"mensaje": "Producto creado", "data": nuevo}

@app.get("/productos")
async def listar_productos():
    return {"total": len(db_productos), "productos": db_productos}
```

Análisis del Código Fuente

Endpoints asíncronos decorados con FastAPI, validación automática mediante Pydantic DTO y códigos de estado HTTP semánticos.

4. Gotchas y Seguridad en Aplicaciones Web

Vulnerabilidades y errores de arquitectura en APIs de producción:

⚠ Gotcha Frecuente (Trampa de Principiante)

Olvidar configurar el middleware CORS (Cross-Origin Resource Sharing), bloqueando las peticiones del frontend.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

Sin configuración CORS: Streamlit o React no
podrán consumir la API

✓ Patrón Pythonic / Correcto

```
from fastapi.middleware.cors import  
CORSMiddleware  
app.add_middleware(CORSMiddleware, allow_origins=  
['*'])
```

🛡 Consejo de Resiliencia en Producción

Utiliza uvicorn main:app --reload durante desarrollo y despliega con contenedores Docker en producción.

5. Resumen Ejecutivo y Conclusiones

Dominas el desarrollo de aplicaciones web full-stack profesionales en Python con FastAPI y Streamlit.



Logro Alcanzado en esta Clase

Capacidad para diseñar y desplegar APIs REST escalables con interfaces interactivas para tu portafolio.

Notas del Instructor y Sigüientes Pasos

Acompañamiento personalizado disponible para la arquitectura y el despliegue de tu proyecto final.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Integra autenticación JWT (JSON Web Tokens) en tu API de FastAPI para proteger endpoints sensibles.